PARTE II – PATRONES ARQUITECTÓNICOS

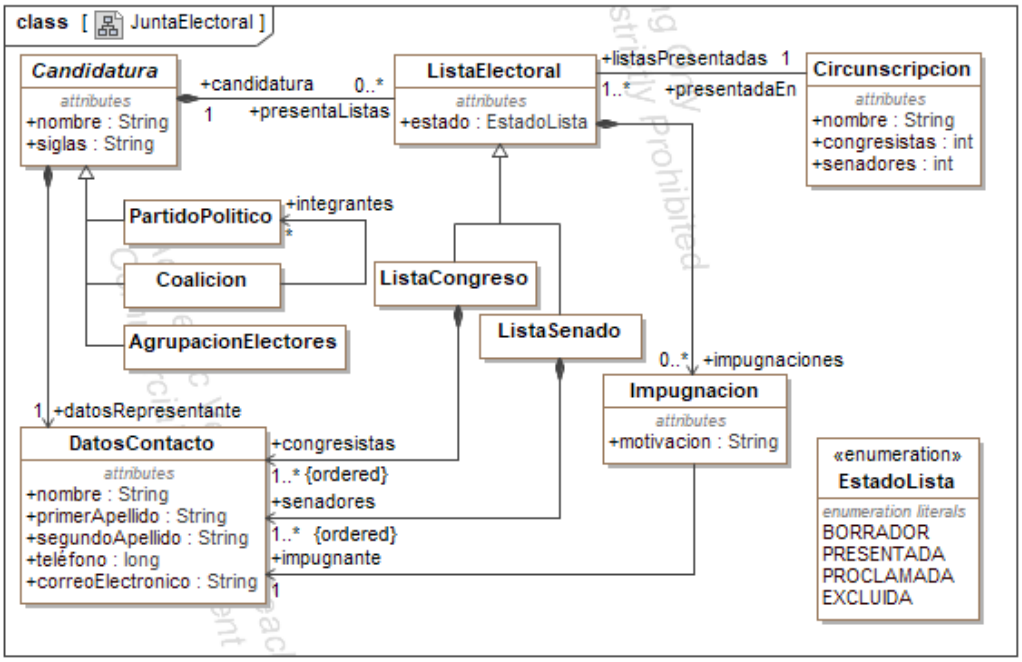


Figura 1. Modelo de dominio para la gestión de listas electorales

Con motivo de la convocatoria de unas elecciones generales en España, la *Junta Electoral Central* desea disponer de una aplicación que ayude a gestionar la presentación, revisión y proclamación de las listas electorales. Para ello, se ha creado un modelo de dominio como el de la Figura 1.

La junta podrá recibir listas de distintas *candidaturas*, que pueden ser *partidos políticos*, *coaliciones* de dos o más partidos o *agrupaciones de electores*. Cada candidatura puede presentar, como máximo, una lista por circunscripción y por cámara. Las listas pueden ser de dos tipos: *listas al congreso*, cerradas y bloqueadas, donde el orden de los candidatos lo fija la candidatura y no puede alterarse; y *listas al senado*, donde se presentan candidatos individuales hasta un máximo establecido por la circunscripción.

Cada circunscripción tiene asociados un número de escaños al congreso de los diputados y un determinado número de senadores. En el caso de las listas al congreso, estás pueden incluir más candidatos que escaños disponibles, entendiéndose que todos aquellos candidatos que superen el número de escaños disponibles son candidatos suplentes, por si los electos no puedan ejercer su función por el motivo que sea.

Una lista nace en estado *borrador*, pasa a *presentada* cuando la candidatura la registra y, tras ser revisada por la *Junta Electoral* correspondiente, puede quedar *proclamada* o *excluida*. Cualquier lista presentada puede recibir *impugnaciones*, que serán consideradas por la Junta Electoral antes de su aprobación.

Dado dicho modelo y teniendo en cuenta que, para todas las operaciones que requieran acceso a la base de datos, las conexiones a la misma se extraerán del pool de conexiones creado en prácticas, se solicita realizar las siguientes acciones utilizando para ello el lenguaje Java:

1. Para las clases *Candidatura*, *ListaCongreso* y *DatosContactoPersona* de la Figura 1, de acuerdo con los principios de la filosofía *Domain-Driven Design*, indique si se tratan de una *Entity* o de un *Value Object*. Proporcionar la respuesta en un archivo *ddd.txt* sito en la carpeta *src/main/resources*. Justificar adecuadamente la respuesta mediante la aportación de ejemplos o evidencias que den soporte a las afirmaciones realizadas. (1.00 punto)
2. Identificar un *aggregate* del modelo de dominio, indicando cuál es su *aggregate root*. Proporcionar la respuesta dentro del archivo *ddd.txt* sito en la carpeta *src/main/resources*. Justificar adecuadamente la respuesta mediante la aportación de ejemplos o evidencias que den soporte a las afirmaciones realizadas. (1.00 punto)
3. Crear un modelo relacional que permita persistir las clases *Candidatura*, *PartidoPolitico*, *Coalicion*, *AgrupacionElectores* y *DatosContacto*, especificando de manera clara y precisa los patrones de transformación aplicados y justificando el patrón seleccionado para transformar la jerarquía de herencia. Proporcionar la respuesta en un archivo *createDatabase.sql* sito en la carpeta *src/main/resources*. (1.00 punto)
4. Implementar, modificando el modelo de dominio donde sea necesario, una operación de servicio que permita a una candidatura añadir un candidato a una lista al congreso. El candidato se añadirá siempre al final de la lista y la operación devolverá como resultado un valor de un enumerado con valores {EXCLUIDO, TITULAR, SUPLENTE} que indicará o que no se ha podido añadir al candidato, o que se ha añadido como titular o que se ha añadido como suplente, respectivamente. Además, la operación debe asegurar que la lista esté en estado *borrador*. Indique dentro del propio método de servicio todos los cambios realizados en el código. Indique también mediante comentarios los puntos concretos del código donde se invocaría a la *Unit of Work*, así como los objetos que se registrarían en ella. Crear *mocks* para simular las clases y métodos de acceso a los datos que se consideren necesarios, dejando su implementación vacía. (1.00 punto)
5. Diseñar los DTO que sean necesarios, e implementar sus correspondientes Assemblers, para mostrar los datos que aparecen en la Figura 2, teniendo en cuenta que la cabeza de lista es el candidato titular en posición 1 y que una lista sin puestos de titular no tiene cabeza de lista. (0.75 puntos)
6. Dada la Figura 2, implementada siguiendo el patrón SPA, ¿qué llamadas se realizarían al servidor al acceder a esta vista desde el inicio de la aplicación? Justifique su respuesta. Proporcionar la respuesta al final del archivo ddd.txt sito en la carpeta src/main/resources. (0.50 puntos)
7. Implementar un proxy para *ListaCongreso* donde todas las referencias de multiplicidad superior a 1 se carguen utilizando el patrón *Lazy Load*. Si se requiriese de la implementación de ciertas operaciones en otros *DataMappers* del sistema, bastará con crear las cabeceras de los métodos que sean necesarios, pero su implementación podrá dejarse vacía. (0.75 puntos)

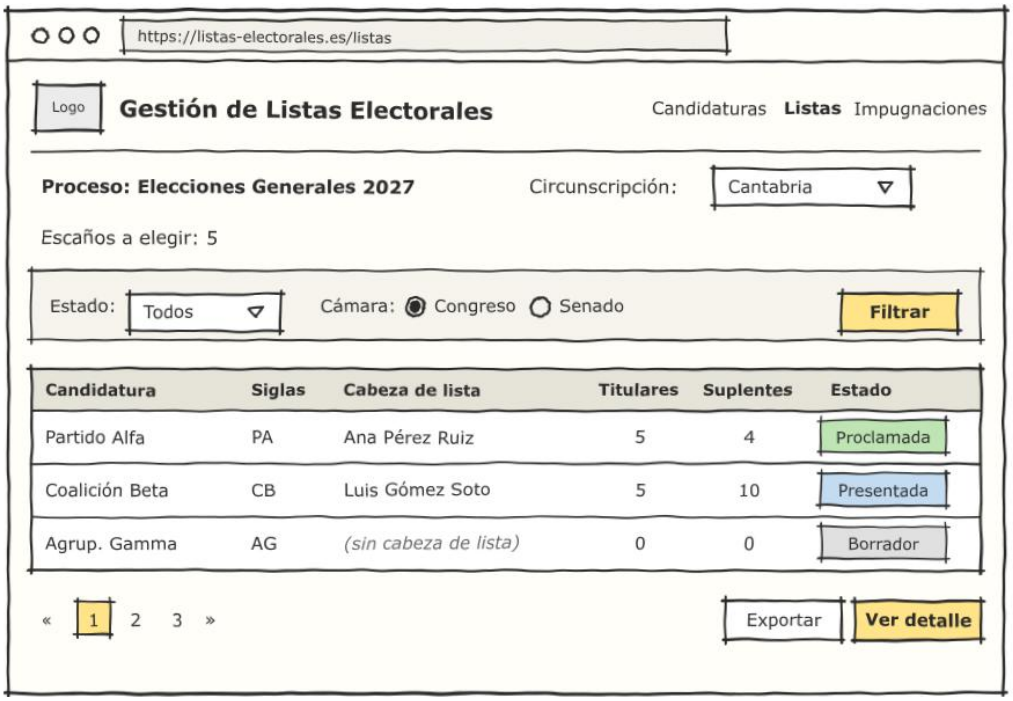


Figura 2. Interfaz para la visualización del estado de las listas electorales

8. Entregar el repositorio creado en la actividad 4 de la práctica correspondiente a la capa de persistencia (práctica 10). (3.00 puntos)
9. Entregar el generador de claves creado en la actividad 3 de la práctica correspondiente a la capa de persistencia (práctica 10). (1.00 punto)

NOTA: Las clases correspondientes al modelo de dominio se pueden descargar ya implementadas de la plataforma Moodle.

Los puntos 1 a 8 y los puntos 9 y 10 se entregarán en archivos *.zip* separados en las tareas habilitadas en *Moodle* a tal efecto. Como nombre para cada archivo *.zip* se utilizará *<ApellidosNombre>.zip*, donde *<ApellidosNombre>* debe sustituirse por los apellidos y el nombre de cada estudiante.

Para poder superar la prueba es necesario obtener una calificación mínima de: (1) 2.25 puntos en los apartados 1 a 8 de esta prueba; o, (2) 4.50 entre los apartados 1 a 8 de esta prueba y los apartados 1 a 7 de la parte correspondiente a patrones de diseño, en el caso de que sea necesario realizarla.

Pablo Sánchez Barreiro